

EventVLA, read closely: turning “when should I remember this?” into a learnable prediction

Paper: EventVLA: Event-Driven Visual Evidence Memory for Long-Horizon Vision-Language-Action Policies

Authors: Ganlin Yang, Zhangzheng Tu, Yuqiang Yang, Sitong Mao, Junyi Dong, Tianxing Chen, Jiaqi Peng, Jing Xiong, Jiafei Cao, Jifeng Dai, Wengang Zhou, Yao Mu, Tai Wang (USTC / Shanghai AI Lab / Huawei / HKU / Tsinghua / PKU / SJTU)

arXiv: [2606.20092](#) (2026-06-18) · Code and data: [InternRobotics/EventVLA](#)

In this repo: [English PDF](#) · [Chinese PDF](#) · [中文解读](#)

0. One sentence

EventVLA lets the VLA policy predict which of the next 50 steps will turn out to be worth remembering, stores only those raw frames in a five-slot buffer, and feeds them back into the vision encoder together with the first frame and a short recent window. On the authors’ new RoboTwin-MeM benchmark this lifts average success from 18.0% (anchors only) to 75.2%; on RMBench the anchors alone reach 67.8% against 42.0% for the previous best (Mem-0); on four real bimanual tasks it scores 90 / 60 / 90 / 75%.

1. The problem

A standard VLA is Markovian, $a_t = \pi(o_t, l)$, which quietly assumes everything relevant stays visible. Real manipulation violates this constantly: the robot lifts a cover, glimpses a colour, closes the cover; reads a random number card; watches a stick point out an order. The evidence appears for a moment and is needed hundreds of steps later. The paper calls these non-Markovian manipulation tasks and sorts existing memory-augmented approaches into three families, each with a concrete failure mode:

Family	Examples	Failure mode
Dual system (high-level VLM manages memory, low-level policy acts)	MemER, Mem-0, MEM (π_MEM)	High latency, errors propagate across the hierarchy
Recurrent / hidden-state compression	AVA-VLA, Recurrent Memory Transformer	Information bottleneck discards fine visual detail
Frame buffers (keep past raw frames)	MemoryVLA, CronusVLA, ContextVLA, LoLA	Unselective accumulation drowns sparse evidence in redundancy and costs compute

The question the paper sets itself: **exactly when, and what, should a VLA preserve so that it succeeds without overwhelming its compute budget?**

2. Method

2.1 Memory structure: anchors plus event frames

The policy becomes $a_t = \pi(o_t, M_{t-1}, l)$ with $M_t = A_t \cup E_t$:

- **Foundational visual anchors** $A_t = \{o_0\} \cup \{o_{t-K}, \dots, o_{t-1}\}$. The initial frame fixes the invariant global layout; the short window supplies motion and progress cues. Rule-based, nothing to learn. Simulation uses o_0, o_{t-30}, o_{t-15} ; the real robot uses $o_0, o_{t-60}, o_{t-40}, o_{t-20}$.
- **Event keyframes** E_t , written by the KEM module, capped at $N_{\max} = 5$, FIFO eviction.

At inference the three parts are concatenated in temporal order, $I_{\text{input}} = \text{concat}([A_t, E_{t-1}, o_t])$, and pushed straight through the VLM vision encoder. There is no separate read module: reading is ordinary multi-frame self-attention.

2.2 KEM: predicting future keyframe probabilities

KEM is a light MLP head next to the action head. Its input is the final-layer hidden state $h_t \in \mathbb{R}^{H \times d}$ of the autoregressive transformer ($H = 50$, the action chunk); its output is one probability per future step:

$$\hat{p}_t = \sigma(\text{KEM_mlp}(h_t)) = [\hat{p}_t^1, \dots, \hat{p}_t^H] \in [0, 1]^H$$

$\hat{p}_t^i$ is the probability that step $t+i$ is a task-critical keyframe. Predicting over the whole chunk rather than step by step matters because events often appear and vanish midway through a chunk, where a per-step classifier would be too late. Sharing h_t is the key design decision: the hidden state already encodes observation and action queries, so the head knows what the policy is about to do and can “schedule” a write in advance.

Write rule: when $\hat{p}_t^i \geq \tau_{\text{commit}}$ (0.55), the raw image at $t+i$ is committed to E_t . Two post-processing steps keep one event from flooding the buffer with near-duplicate frames:

1. **1-D NMS** keeps only local maxima within a window of radius $w = 8$;
2. **Cooldown** drops any candidate less than $C = 10$ steps after the last committed frame.

2.3 Training: automatic labels, soft targets, teacher-to-student curriculum

- **Labels** come from Qwen3-VL-235B-A22B (8× A800, vLLM) watching demonstrations offline: 128 uniformly sampled multi-view frames per episode, few-shot prompted to emit keyframe steps as JSON. Against physics-engine ground truth in simulation the mean temporal error is under 10 steps; against human labels on the real robot it stays within 50 steps (episodes of 1500–2000 steps).
- **Soft targets**. Hard 0/1 labels destabilise the head because physical events are temporally fuzzy; the labels are smoothed with a raised-cosine kernel of radius R : $y_t^i = 0.5(1 + \cos(\pi|t+i-t^*|/R))$.
- **Loss**: $L = L_{\text{action}} + \lambda L_{\text{kem}}$, with L_{kem} a sequence-averaged BCE and $\lambda = 0.1$.

- **Curriculum.** Early in training the buffer is built from ground-truth keyframes (teacher forcing) with probability α , which decays linearly from 1 to 0 so the policy ends up relying on its own thresholded predictions. This closes the train/test gap.

2.4 Backbone and hyper-parameters

Setting	RMBench / RoboTwin-MeM	Real robot
Base	QwenOFT (Qwen3-VL-4B-Instruct, StarVLA codebase)	$\pi_{0.5}$ (PaliGemma)
Action head	OFT, $H = 50$, 14-D actions	flow head, $H = 50$, 32-D actions
Training	80k steps, VLM lr $1e-5$, new modules $1e-4$	60k steps, $5e-5$ cosine, global batch 32
Images	224×224	224×224
KEM	$\tau=0.55$, $N_{\max}=5$, $w=8$, $C=10$, $\lambda=0.1$	same

3. The RoboTwin-MeM benchmark

The authors argue that RMBench and RoboMME can mostly be solved from the first frame plus a recent window, so evidence that appears only transiently mid-interaction has never been isolated. RoboTwin-MeM (RoboTwin 2.0, SAPIEN) has eight bimanual tasks, each tagged with $n \in [1, 5]$, the number of intermediate keyframes that must be retained:

Task	n	Avg. steps	What is tested
Rearrange Blocks Hard	1	879	Which block was moved
Put Back Block Hard	2	1468	Which outer pad each block visited
Pick Objects in Order / Pick the Unhidden Block	3	1124 / 699	Lift covers, close them, pick by order / by elimination
Cover Blocks Hard / Find Seal Stamp / Reproduce Route	4	1544 / 1338 / 1417	Remember colours across covers; return the seal under its cover; copy a demonstrated route (in-context)
Press Button Keyframe	2–5	430	Read two number cards, press buttons that many times (counting)

Three capabilities are separated: transient evidence memory (cover tasks), sequence and counting (buttons), and in-context imitation (Reproduce Route). Each task has 50 demonstrations with per-step language annotations.

4. Results

4.1 RMBench: anchors are enough

Method	Avg. success
$\pi_{0.5}$ / X-VLA / QwenOFT (no memory)	10.4 / 9.8 / 5.6

Method	Avg. success
MemER / Mem-0 (dual system)	8.7 / 42.0
MemoryVLA (OpenVLA) / MemoryVLA (QwenOFT)	19.4 / 41.7
EventVLA without initial frame / without short-term window	33.7 / 23.8
EventVLA (visual anchors only)	67.8

RMBench’s nine tasks depend on persistent layouts and fixed motion styles, so only the anchor version was deployed. Both anchor components are necessary: removing either drops the score to 33.7% or 23.8%. Per task, Rearrange Blocks 96%, Put Back Block 95%, Swap Blocks 96%, Cover Blocks 97%, but Observe and Pick Up 21% and Press Button 3%: counting is beyond anchors.

4.2 RoboTwin-MeM: KEM is the qualitative jump

Method	n=1	n=2	n=3	n=3	n=4	n=4	n=4	n=5	Avg.
$\pi_{0.5}$	20	19	1	14	0	8	0	0	7.8
MemER	32	4	12	2	0	26	3	5	10.5
MemoryVLA (QwenOFT)	39	0	1	9	1	11	0	25	10.8
EventVLA anchors only	62	13	5	20	0	26	0	18	18.0
EventVLA anchors + KEM	62	93	90	54	94	63	98	48	75.2

(Columns: Rearrange Blocks Hard, Put Back Block Hard, Pick Objects in Order, Pick the Unhidden Block, Cover Blocks Hard, Find Seal Stamp, Reproduce Route, Press Button Keyframe.)

The entire jump from 18.0% to 75.2% comes from KEM. At n=1 anchors already suffice (62% unchanged); for n≥2 almost all the success comes from event frames. Mem-0 scores 0% on this suite.

4.3 Ablations: what each design choice is worth

Variant	Avg.	Reading
Full EventVLA	75.2	—
Implicit memory bank (keyframes compressed into one latent)	24.9	Several events squeezed into one vector: information bottleneck
Hard labels	48.8	Unstable head, missed writes
No NMS	53.4	Adjacent frames of one event flood the buffer; early evidence evicted
N_max = 2	32.0	Not enough capacity
Chunk 30 / 15	31.1 / 13.6	Foresight window too short to schedule writes

“Implicit bank 24.9 vs raw frames 75.2” is the most consequential number in the paper: it refutes the default assumption that compressing history into a latent is enough, at least for tasks that need three to five distinct visual details at once.

4.4 No regression on Markovian tasks

On standard RoboTwin 2.0 tasks EventVLA improves on its QwenOFT base, Easy 80.0→83.8% and Hard 78.0→81.6%, and edges $\pi_{0.5}$ (82.7 / 76.8).

4.5 Real robot: ARX ACONE bimanual

Four tasks, 20 trials each: Find Block Easy / Hard (the hidden block’s location is only briefly visible), Pick-X-Times (read a random number, count), Pick in Order (a stick points out a sequence).

Method	Find Block Easy	Find Block Hard	Pick-X-Times	Pick in Order
$\pi_{0.5}$	0–10%	0–10%	0–10%	0–10%
π_{MEM} (reproduced)	50	—	30	40
EventVLA	90	60	90	75

4.6 The cost: throughput drops to a third

Method	Latency (s/chunk)	Throughput (chunks/s)
QwenOFT	0.36	2.91
EventVLA anchors only	0.96	1.07
EventVLA anchors + KEM	1.09	0.94

Multi-frame input multiplies the visual tokens; throughput falls from 2.91 Hz to 0.94 Hz. The authors’ defence is that VLAs act as high-level planners over high-frequency low-level controllers, so 1 Hz is workable.

5. Assessment

5.1 The real contributions

1. **“When to write” becomes a prediction instead of a rule.** Previous methods wrote every frame, sampled at fixed intervals, or delegated the decision to an external VLM. KEM predicts write probabilities for the next 50 steps from the policy’s own hidden state, so the policy effectively reserves a memory slot for something it is about to see. It shares the representation with action prediction and adds almost no parameters.
2. **A clean controlled comparison of raw frames versus latents.** 75.2 versus 24.9 was measured inside one framework, on one dataset, changing only the memory representation. That is far more credible than cross-paper comparison.

3. **Memory demand parameterised by n.** RoboTwin-MeM makes “how many things must be remembered” an explicit task attribute, so success can be plotted against n. Earlier memory benchmarks could not offer that diagnostic.

5.2 Points to keep in mind

1. **KEM was never evaluated on RMBench.** The 67.8% there is the anchor-only model; KEM’s gains appear only on the authors’ own RoboTwin-MeM. Both benchmarks come from the RoboTwin ecosystem (RMBench’s first author Tianxing Chen is also an author here), whose design naturally favours the anchors-plus-keyframes hypothesis. No RoboMME or MIKASA-Robo numbers are reported.
2. **KEM is tightly coupled to long action chunks.** Shrinking the chunk from 50 to 15 collapses success from 75.2% to 13.6%, below the anchors-only model (18.0%). Foresight is a function of chunk length, so the mechanism will not transfer directly to tasks that need short chunks and fast reaction (contact-rich or dynamic scenes).
3. **A capacity-5 FIFO ceiling.** The limitations section concedes that tasks over 10 minutes with dense events will saturate the buffer and evict early evidence. Even within the benchmark, the hardest $n=5$ task scores 48% and Find Seal Stamp ($n=4$) 63%: success clearly declines with n.
4. **Labelling depends on a 235B model.** Running Qwen3-VL-235B on eight A800s for offline annotation is a cost the paper does not quantify. Real-robot keyframe error of up to 50 steps is absorbed by the soft labels.
5. **A threefold latency penalty.** 0.94 Hz is fine for tabletop bimanual tasks and a real cost for mobile manipulation or dynamic scenes. NativeMEM and TempoFit attack precisely this axis (one token per frame; no new tokens, reuse KV).
6. **20 real trials per task**, no confidence intervals; π_MEM is the authors’ reproduction.

5.3 EventVLA versus TRACE

Both papers address the same situation (the evidence is gone by decision time) and take opposite positions on three axes:

Axis	EventVLA	TRACE
Memory content	Raw image frames	512-D latent slots
Write trigger	Learned future-keyframe probability + NMS + cooldown	Write every step; signature-routed, gated updates decide how much and where
Addressing	Temporal concatenation, attention finds what it needs	Path signatures of the robot-state trajectory as keys
Capacity	5 frames + anchors	$K = 4$ or 6 fixed slots
Relation to backbone	End-to-end, enters the VLM vision encoder	Adapter; backbone, action head and loss untouched

Axis	EventVLA	TRACE
Inference overhead	Throughput 2.91→0.94 Hz	+2.1 ms per step

The designs match two kinds of information: **high-dimensional visual detail** (colours, positions, several objects) versus **low-dimensional branch variables** (which source, which route). EventVLA’s implicit-bank ablation (24.9%) shows the former will not compress into a vector; TRACE’s results show the latter fits in a few slots. Which memory to choose depends on how many bits the downstream decision must recover.

6. Related reading

- **KEMO** ([2606.23589](#), same month): event-driven keyframe memory whose events come from robot kinematics plus visual filtering rather than a learned head; keyframes become tokens fused by cross-attention and gated residuals. Real-robot task success +23.6% over $\pi_{0.5}$. Two near-simultaneous papers make event-driven sparse writing the consensus direction of mid-2026.
- **UniMem** ([2608.22869](#)): one backbone with an event classifier for memory updates and a keyframe encoder for dense spatial memory; 93.4% vs 68.2% for fixed-interval sampling in simulation.
- **MemoryWAM** ([2606.20562](#)): recent frames + event-boundary anchor frames + gist tokens inside a world action model, structurally the same anchors-plus-events layout.
- **MemER** ([2510.20328](#)): a high-level VLM selects keyframes in a dual system; used here as the dual-system baseline (10.5% on RoboTwin-MeM).
- **RM Bench** ([2603.01229](#)) and **RoboMME** ([2603.04639](#)): the benchmarks EventVLA argues are solvable by anchors; RoboTwin-MeM fills the transient-evidence gap.
- **Present but Not Remembered** ([2607.03372](#)): probing shows history inside frozen VLAs is largely a redundant copy of the present and recommends injecting information unique to the past, which is exactly what sparse event frames do.